

NOTES

Function



Functions

1. Introduction

A function is a block of reusable code that performs a specific task. Functions help organise code into manageable sections, making it easier to read, maintain, and debug. They can accept inputs (known as parameters or arguments), perform operations, and return outputs.

1.1 What is a Function?

A function is a named block of reusable code that performs a specific task. Functions help organise code into manageable sections, making it easier to read, maintain, and debug.

1.2 Why Use Functions?

Reusability: Functions allow you to write code once and reuse it multiple times.

Modularity: Functions break down complex problems into smaller, manageable parts.

Readability: They make the code more organised and easier to understand.

Maintainability: Changes can be made in one place without affecting other parts of the code.

1.3 Defining a Function

You define a function using the `def` keyword followed by the function name and parentheses containing any parameters. The body of the function is indented.

```
Python
def add_numbers(a, b):
    """Return the sum of two numbers."""
    return a + b

# Calling the function
result = add_numbers(5, 3)
print(result) # Output: 8
```

2. Function Parameters

Functions can accept parameters, which are the inputs you provide to the function. There are several types of parameters:

2.1. Positional Parameters:

The order in which you provide arguments matters.

```
Python
def greet(name):
    return f"Hello, {name}!"
print(greet("Alice")) # Output: Hello, Alice!
```

2.2. Default Parameters:

We can easily set default values for parameters.

```
Python
def greet(name="Guest"):
    return f"Hello, {name}!"

print(greet()) # Output: Hello, Guest!
print(greet("Bob")) # Output: Hello, Bob!
```

2.3. Keyword Parameters:

We can easily specify parameters by name when calling the function.

```
Python
def describe_person(name, age):
    return f"{name} is {age} years old."

print(describe_person(age=30, name="Alice"))
# Output: Alice is 30 years old.
```

2.4. Variable-length Arguments:

We can easily use `\*args` for a variable number of positional arguments and `kwargs` for a variable number of keyword arguments.

```
Python
def add_all(*args):
    return sum(args)

print(add_all(1, 2, 3, 4)) # Output: 10

def print_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_info(name="Alice", age=30)
# Output: name: Alice \n age: 30
```

2.5 Return Statement

The `return` statement is used to send a result back from the function. If no return statement is specified, the function will return `None`.

Example of a Function with Return

```
Python
def multiply(x, y):
    """Return the product of x and y."""
    return x * y

print(multiply(2, 3))          # Output: 6
```

2.6 Scope of Variables

Variables defined inside a function are local to that function and cannot be accessed from outside (local scope). Variables defined outside any function have global scope and can be accessed anywhere.

```
Python
x = 10 # Global variable

def my_function():
    y = 5 # Local variable
    return x + y # Can access global variable x

print(my_function()) # Output: 15
# print(y) # Raises NameError: name 'y' is not defined
```

3. Iterators

Iterators in Python are objects that allow you to traverse through a collection (like a list, tuple, or dictionary) one element at a time without exposing the underlying details of the collection. They are an essential concept for handling loops and data processing in Python.

3.1 Key Concepts

3.1.1. Iterator:

An object that implements two methods: `__iter__()` and `__next__()`.

- `__iter__()`: Returns the iterator object itself. This is used in cases where an iterable needs to be converted to an iterator.
- `__next__()`: Returns the next value from the iterator. If there are no more items to return, it raises a `StopIteration` exception.

3.1.2. Iterable:

An object that can return an iterator. It implements the `__iter__()` method. Examples include lists, tuples, sets, dictionaries, and strings.

3.1.3. For Loops:

In Python, for loops automatically call the iterator protocol, making it easy to iterate over iterable objects.

3.2 Creating an Iterator

We can easily create your own iterator by defining a class that implements the `__iter__()` and `__next__()` methods.

Example of a Custom Iterator: Here's an example of an iterator that counts from 1 to a specified number:

```
Python
class Counter:
    def __init__(self, limit):
        self.limit = limit
        self.current = 0

    def __iter__(self):
        return self

    def __next__(self):
```

```

        if self.current < self.limit:
            self.current += 1
            return self.current
        else:
            raise StopIteration

# Using the Counter iterator
counter = Counter(5)

for number in counter:
    print(number)
# Output: 1, 2, 3, 4, 5

```

3.3 Using Built-in Iterators

Python provides several built-in iterators, which we can easily use without needing to create your own. For example:

3.3.1 List Iterators:

```

Python
my_list = [1, 2, 3, 4, 5]
list_iterator = iter(my_list)

print(next(list_iterator)) # Output: 1
print(next(list_iterator)) # Output: 2

```

3.3.2 Tuple Iterators:

```

Python
my_tuple = (1, 2, 3)
tuple_iterator = iter(my_tuple)

for item in tuple_iterator:
    print(item) # Output: 1, 2, 3

```

3.3.3 Dictionary Iterators:

```
Python
my_dict = {'a': 1, 'b': 2, 'c': 3}
dict_iterator = iter(my_dict)

for key in dict_iterator:
    print(key)
# Output: a, b, c (the order may vary)
```

4. Generator Functions

A simpler way to create iterators is by using **generator functions**. A generator is a special type of iterator that uses the **yield** keyword instead of **return**. When a generator function is called, it returns a generator object without executing the function immediately.

4.1 Creating a Generator Function

Here's an example of a generator that yields numbers from 1 to a specified limit:

```
Python
def count_up_to(limit):
    count = 1
    while count <= limit:
        yield count
        count += 1

# Using the generator
for number in count_up_to(5):
    print(number) # Output: 1, 2, 3, 4, 5
```

4.2 A generator function to generate Yields

```
Python
def simple_generator():
    yield 1
    yield 2
    yield 3

# Using the generator
gen = simple_generator()

print(next(gen)) # Output: 1
print(next(gen)) # Output: 2
print(next(gen)) # Output: 3
```

5. Lambda Functions

Lambda functions in Python are small, anonymous functions defined using the `lambda` keyword. They are often used for short, throwaway functions that are not intended to be reused later in the program. Lambda functions can have any number of arguments but only one expression, which is evaluated and returned.

5.1 Key Characteristics of Lambda Functions

- **Anonymous:** Lambda functions are defined without a name, unlike regular functions defined using the `def` keyword.
- **Single Expression:** A lambda function can contain only a single expression. This means it cannot contain commands or multiple expressions.
- **Return Value:** The result of the expression is returned automatically, without the need for a `return` statement.

lambda arguments: expression

- **arguments:** The parameters the function accepts (can be zero or more).
- **expression:** A single expression that is evaluated and returned.

5.2 Basic Lambda Function

Here's a simple example of a lambda function that adds two numbers:

```
Python
add = lambda x, y: x + y
result = add(3, 5)
print(result) # Output: 8
```

5.3 Using Lambda with Built-in Functions

Lambda functions are commonly used with functions like ``map()``, ``filter()``, and ``reduce()``.

5.3.1. Using ``map()``:

The ``map()`` function applies a given function to all items in an iterable (like a list).

```
Python
numbers = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x**2, numbers))

print(squared) # Output: [1, 4, 9, 16, 25]
```

5.3.2. Using ``filter()``:

The ``filter()`` function filters elements from an iterable based on a function that returns ``True`` or ``False``.

```
Python
numbers = [1, 2, 3, 4, 5]
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
```

```
print(even_numbers) # Output: [2, 4]
```

5.3.3. Using `reduce()`:

The `reduce()` function from the `functools` module applies a rolling computation to sequential pairs of values in a list.

```
Python
from functools import reduce

numbers = [1, 2, 3, 4]
product = reduce(lambda x, y: x * y, numbers)

print(product) # Output: 24
```

5.4 Advantages of Lambda Functions

- **Conciseness:** Lambda functions allow you to write small functions in a single line, which can improve the readability of the code.
- **Convenience:** They can be used for short-lived operations without the need to define a full function.
- **Functional Programming:** Lambda functions are essential for functional programming techniques in Python, allowing for higher-order functions and passing functions as arguments.

5.5 Limitations of Lambda Functions

- **Single Expression:** Since lambda functions can only contain a single expression, they are limited in functionality compared to regular functions.
- **Less Readable:** While they can make code shorter, using lambda functions excessively can lead to less readable code, especially for complex operations.

- **No Documentation:** Lambda functions do not have a name, which makes it difficult to document them or understand their purpose quickly.

Example:

```
Python
from functools import reduce

numbers = [1, 2, 3, 4, 5]

# Square the numbers
squared = list(map(lambda x: x ** 2, numbers))

# Filter the even numbers
even_squared = list(filter(lambda x: x % 2 == 0, squared))

# Reduce to get the product of even squares
product = reduce(lambda x, y: x * y, even_squared)

print(product) # Output: 64 (4*16)
```

5.6 Conclusion

- `map()` is used to apply a function to all elements of an iterable.
- `filter()` is used to filter elements based on a condition.
- `reduce()` is used to reduce an iterable to a single value by applying a function cumulatively.

These functions promote a functional programming style, allowing for clean and concise data processing in Python.

THANK YOU

